Decoder capacity, not dopamine or label-guided growth, explains category decoding in a recoded *Drosophila* mushroom-body subcircuit

A FlyWire model with growth-matched controls

Nicolas Giavedoni

Draft manuscript

Abstract

We use a fixed subcircuit of the adult *Drosophila melanogaster* connectome (673 antennal-lobe projection neurons → 384 Kenyon cells → 4 mushroom-body output neurons, with **zero artificial edges**) to classify four-letter words drawn from four categories. Words are encoded as seeded per-category prototype patterns over the input channels, not as orthographic or semantic forms. At the physiological operating point the Kenyon-cell population carries no linearly decodable class code (held-out probe 0.255) although the stimulus does (probe 1.000); raising the gain of the `ALPN → Kenyon cell` transform (a "recode", to 20 or 50, with the calibration rate ceiling widened to 100 Hz) restores the code (probe 0.950–1.000) and the network then learns.

We are explicit about what the result is and is not. The **untrained network already scores 0.33**, so we report significance against that internal floor rather than against chance (0.25): the plain trained condition is only marginally above it ($p = 0.02$). The online MBON-argmax readout stays at chance (≈0.25) in every condition except the supervised-pretrained one, so the reported accuracy is a statement about the representation the network builds, not about an online learner.

The largest supervised-decoder numbers come from structural growth (`structural_on` 0.833 on the main recode run, 0.750 on the matched-control run; growth-off conditions 0.378–0.435; paired $p < 0.001$), and switching dopamine off within the growth-on regime changes nothing ($p = 0.19$ / 0.084). We then ran the growth-matched controls. **They demote the growth claim.** The gain is unchanged when all reward/label guidance of growth is removed (`structural_unguided` 0.785, paired $p = 0.75$) or category inheritance is broken (`structural_random_category` 0.707, $p = 0.61$), and it is **exceeded by a no-growth decoder given the same number of extra feature dimensions (300 random Kenyon-cell projections: 0.983, $p = 0.002$ vs `trained`).** The growth effect is therefore best explained as expanded decoder access to an already decodable Kenyon-cell code — a **decoder-capacity effect** — rather than as a learned, label-guided class prototype, and it is not evidence of online learning.

Three further departures bound the claim: the readout is a supervised linear decoder fitted *after* training, and the online MBON-argmax readout stays at chance (≈0.25) throughout; the 100 Hz calibration ceiling lies far outside the physiological 0.5–15 Hz band; and the extracted `DAN → Kenyon cell` edges are not actually simulated. We also document a decoder-fit leak we introduced

and fixed: a shuffled-label control that briefly scored 0.615 now sits at or below the untrained floor. The positive contribution is a causal, well-controlled localisation — the `ALPN → Kenyon cell` transform, not the plastic readout, is the gate on whether this circuit's class code survives — together with a reproducible pipeline in which the result can be inspected and challenged, and a set of released growth-matched controls that demote the paper's own headline.

1. Introduction

The mushroom body of the insect brain is a canonical substrate for associative learning: sparse, high-dimensional Kenyon-cell representations of odour are read out by mushroom-body output neurons (MBONs) whose synapses are modified by dopaminergic reinforcement neurons (DANs) [Aso et al., 2014; Hige et al., 2015]. Whole-brain connectomes at synapse resolution [Zheng et al., 2018; Scheffer et al., 2020; Dorkenwald et al., 2024] now make it possible to ask whether that circuit, instantiated at its measured synaptic structure, can actually *learn a task* under a biologically motivated plasticity rule.

We pose a small, fully specified classification problem: assign four-letter English words to four semantic categories (animal, food, tool, place). The problem is small enough to run in minutes per condition on a laptop and large enough to have genuine class structure. The scientific question is not whether a larger or better-tuned model can win, but whether the *real wiring* plus a simple, documented rule can learn — and if so, what does the learning.

The study answers with a mechanism, not just a number. The real olfactory chain scores above its untrained floor once the gain of the `ALPN → Kenyon cell` transform is raised; at the physiological operating point the class code is destroyed at that transform, before the plastic readout ever sees it. Within the recoded regime structural growth produces the largest decoder accuracy, but the growth-matched controls show that this is a decoder-capacity effect — it does not require label guidance and is beaten by plain random features — and that the online readout never learns. We therefore report a mechanism, an unusually explicit account of how fragile the numbers are and what they rest on, and the controls that demote the paper's own headline. Section 2 gives background. Section 3 details data, subcircuit, encoding, network, learning rule, recode, and evaluation protocol. Section 4 reports the result, the causal localisation with probes, the growth-versus-dopamine decomposition, and the controls. Section 5 discusses mechanics, threats to validity, and what would change the answer. Section 6 concludes.

2. Background

Connectome. FlyWire release 783 provides a proofread synapse-level wiring diagram of an adult *Drosophila* brain [Dorkenwald et al., 2024]. We use it only as a connectivity graph: nodes are neurons, directed weighted edges are synapse counts, and cell classes (e.g. `ALPN`, `Kenyon_Cell`, `MBON`, `DAN`) come from the accompanying annotations [Schlegel et al., 2024].

Mushroom body. Kenyon cells receive predominantly olfactory input from antennal-lobe projection neurons and give sparse representations; MBONs read this population out; DANs carry reinforcement and gate plasticity at the Kenyon-cell → MBON synapses [Aso et al., 2014].

Learning is widely modelled as a three-factor rule: a Hebbian eligibility trace multiplied by a modulatory (reward) signal [Hebb, 1949; Schultz et al., 1997].

Simulation. We use Brian2 [Stimberg et al., 2019] with leaky integrate-and-fire neurons and a per-run no-stimulus calibration. Every run writes a manifest with the resolved configuration, seed state, data checksums, calibration, subcircuit statistics, codegen target, hardware, and git commit.

3. Methods

3.1 Data and subcircuit extraction

The pipeline verifies and loads the FlyWire proofread connections (`data/flywire/proofread_connections_783.feather`, 16.8 M rows), maps neurons to a stable index, assigns connection signs, and extracts a subcircuit by cell class. Output neurons are four MBONs. The configuration selects the olfactory pathway:

```
ALPN (olfactory input) → Kenyon cell (mushroom body) → MBON (output)
                                      ↑
                                      |
                                DAN (reinforcement)
```

Measured connectivity used (edges / synapses) is ALPN⟶KC 28,144 / 329,394; KC⟶MBON 89,315 / 256,719; DAN⟶KC 49,616 / 60,657. The extracted subcircuit contains **1,065 neurons, 23,075 edges and 120,424 synapses with zero artificial edges** (673 ALPN inputs, 384 Kenyon cells, 4 DAN, 4 MBON). Because a global weight scale is not identifiable a priori, the network is calibrated once per run to a spontaneous regime; the accepted scale was 4.0 (noise weight 4.0, 6 Hz background).

3.2 Stimulus encoding

The task assigns 4-letter words to four categories, but the words are **not** processed as letters or meaning. Each category owns a fixed, seeded, disjoint subset of the input neurons; each word drives 64 inputs from its own category prototype plus 4 cross-category inputs at graded drive in [0.35, 1.0], with the rest silent. The stimuli are therefore **category prototype patterns with partial cross-category overlap** — a deliberately easy, low-dimensional task. "Word" below denotes the token that indexes a prototype; no orthographic or semantic structure is present. The mapping uses the selected input-stage size (673 inputs).

3.3 Network and plasticity

The network is a population of LIF neurons wired by the extracted edges. Plasticity is restricted to the Kenyon-cell ⟶ MBON synapses. Each synapse carries a decaying eligibility trace updated by pre/post activity; the weight update is the trace multiplied by a dopamine signal and a learning rate, then clipped. Dopamine is delivered per class: on each trial the four MBONs receive credit according to whether their class is correct, so the rule is a per-class reward. A `dopamine off` switch disables the multiplicative factor while leaving the trace dynamics intact.

Structural plasticity adds a reserve pool of silent, unconnected neurons (1000; 700 available, 300 activated). At fixed intervals, reserve neurons are recruited near the most successful readout unit,

inherit its category and at most (1 – divergence) of its inputs, and draw the remainder pseudo-randomly; deletions retire the weakest mature reserve synapses so they track a configured fraction of additions. The mechanism is exploratory — it has no published precedent — and is switched on/off as an ablation. **The recruitment rule is supervised by the network's own predictions and transfers the winning unit's category, so it may construct a class prototype directly rather than learn one; Section 5.2 makes this the central threat to the interpretation of the headline number.**

3.4 Recoding the input transform

The `ALPN → Kenyon cell` projection carries a configurable pair gain (default 1.0, the identity). The recode raises this gain to 20 or 50 and widens the calibration rate ceiling (`calibration.max_mean_rate_hz`, default 15 Hz) to 100 Hz, so that calibration may accept a scale at which the Kenyon cells carry the label (the single global weight scale otherwise falls as the gain rises, holding Kenyon-cell activity roughly constant and nullifying the gain). **The 100 Hz ceiling is far outside the physiological 0.5–15 Hz band**; the recode is a diagnostic intervention that isolates the transform as the bottleneck, not a claim about how the circuit operates *in vivo*.

3.5 Readout and evaluation

The reported readout is a **supervised linear decoder fitted on a fixed post-training feature pass** — a specified, documented departure from an online MBON readout that isolates *representation* quality from *decoding* (an online MBON-argmax readout is also produced and is no better; Section 5.3).

Representation probes. To separate "the network cannot learn" from "the evaluation cannot detect learning", two linear probes are fitted on a *fresh untrained* network and scored on held-out words with the same decoder protocol: one on the input stage and one on the Kenyon-cell stage. A probe reports training, k-fold cross-validation, and held-out accuracy per seed and aggregated. A *positive-control gate* passes only if the input probe decodes above 0.90 and the Kenyon-cell probe above chance + 0.05.

Conditions. The trained condition is nominally the main result. Controls are: untrained; shuffled training labels (`shuffled_labels`); dopamine off (`dopamine_off`); structural plasticity on/off (`structural_on`, `structural_off`); supervised-pretrained readout (`trained_pretrained`); and two isolation controls — `dopamine_off_no_growth` and `shuffled_labels_no_growth` — which complete the dopamine × growth and shuffled × growth 2 × 2 read-outs. Note that **`trained` and `structural_off` are the same configuration** (`{"trained": true, "structural": false}`) and are retained as an internal replicate; they are expected to agree, and any disagreement measures residual run-order variance (Section 5.4).

Statistics. Because the untrained network is not at chance (Section 4.1), the correct null for the learning effect is the **untrained internal control**, not chance. We therefore report one-sample tests against chance *and* paired per-seed contrasts against the untrained network and across the ablations. Protocol: seeded 15/5 per-category train/test split; 600 training trials per seed; 95 % percentile-bootstrap CIs over seeds; significance at $\alpha = 0.05$. Formal runs use 10 seeds (1000–1009) with `eval_repeats = 3`.

Leakage margin. The supervised decoder is fitted on the labels each training trial was actually *presented* with. A data-leakage warning is raised only when the shuffled-label control exceeds the higher of chance and the untrained-network floor by a configured margin (0.05).

3.6 Reproducibility

Seeds are independent, so each formal run executes one `evaluate` process per seed (`flyread sweep`), resumes completed (seed, condition) pairs from a checkpoint, and merges the shards into a single report. Kernels were compiled with Cython. All archived runs carry their manifest.

4. Results

4.1 The untrained network is well above chance

Before interpreting any learning effect, the internal floor matters. The untrained network scores **0.338** (gain 20) and **0.330** (gain 50) — a full 8–9 points above the 0.25 chance level (Table 1). This is the recoded representation's own decodability, not learning. Testing against chance therefore inflates every significance value; the meaningful comparison is against the untrained control.

4.2 The recoded network scores above its untrained floor, but the effect is marginal

Table 1 gives the two recoded runs (`runs/recode-v2-merged`, gain 20; `runs/recode-50-v2-merged`, gain 50; 10 seeds each); both pass the positive-control gate and carry no leakage warning.

Table 1: **Table 1.** Supervised-decoder held-out accuracy by condition (10 seeds, chance 0.25), with 95 % percentile-bootstrap CIs. `online argmax` is the MBON-argmax readout — mean accuracy over the training presentations, the quantity that tests the plasticity rule itself rather than the post-hoc decoder; it sits at chance in every condition except the supervised-pretrained control. Untrained floor in bold.

condition	gain 20 decoder (95 % CI)	gain 50 decoder (95 % CI)	gain 50 online argmax
untrained (internal floor)	**0.338** [0.275, 0.393]	**0.330** [0.265, 0.390]	n/a
trained	0.363 [0.308, 0.408]	0.418 [0.367, 0.468]	0.240
structural_off (≡ trained config)	0.370 [0.312, 0.415]	0.435 [0.395, 0.477]	0.239
shuffled_labels	0.330 [0.270, 0.388]	0.245 [0.195, 0.303]	0.246
shuffled_labels_no_growth	0.230 [0.185, 0.288]	0.233 [0.182, 0.282]	0.251
dopamine_off	0.597 [0.532, 0.658]	0.785 [0.708, 0.850]	0.250
dopamine_off_no_growth	0.357 [0.298, 0.415]	0.387 [0.333, 0.435]	0.250
trained_pretrained	0.512 [0.467, 0.553]	0.340 [0.302, 0.380]	0.351
structural_on	**0.608** [0.552, 0.662]	**0.833** [0.785, 0.880]	0.250

Decoded significance against chance (one-sample t): trained $p = 0.0013$ at gain 20 and 8.5×10^{-5} at gain 50, but the honest comparison is the untrained floor (paired $p = 0.39$ and 0.020). The online MBON-argmax readout, by contrast, is at chance in *every* condition (0.239–0.254) except `trained_pretrained` (0.351 at gain 50), whose plastic weights are initialised from a supervised fit. The entire recode effect — including 0.833 — is therefore visible only through the supervised post-hoc decoder: it is a statement about the *representation* the network builds, not about an online learner.

All conditions are shown in Table 6 with the switch settings that define them.

The plain trained condition lies only **+0.025 above the untrained floor at gain 20 (paired p = 0.39, not significant)** and +0.088 at gain 50 (paired $p = 0.020$, marginal). Its apparent significance against chance (0.0013 / 8.5×10^{-5}) does not survive comparison with the internal control. The large, robust effects in Table 1 are the growth-on conditions; we analyse them next.

4.3 The robust effect is structural growth, and it is not dopamine

Reading the 2 × 2 ablations isolates the effective mechanism (Table 2). Accuracy follows the growth switch, not the dopamine switch.

Table 2: **Table 2.** Paired per-seed contrasts (n = 10). Positive Δ means the first condition is higher.

contrast	isolates	Δ acc (gain 20)	p	Δ acc (gain 50)	p
trained – untrained	learning above floor	+0.025	0.39	+0.088	0.020
structural_on – structural_off	**growth effect**	+0.238	< 0.001	+0.398	< 0.001
structural_on – dopamine_off	**dopamine effect**	+0.012	0.75	+0.048	0.19
dopamine_off – dopamine_off_no_growth	growth effect (dop off)	+0.240	0.001	+0.398	< 0.001

- **Growth effect is large and robust.** `structural_on` (0.833) exceeds `structural_off` (0.435) by +0.398 at gain 50 (paired $p < 0.001$) and by +0.238 at gain 20; the same gap appears within the dopamine-off arm.
- **Dopamine effect is negligible.** Within the growth-on regime, switching dopamine off (`dopamine_off` 0.785 vs `structural_on` 0.833) changes accuracy by +0.048 (paired $p = 0.19$; +0.012, $p = 0.75$ at gain 20). The dopamine-gated weight rule contributes essentially nothing once growth is enabled.

Structural plasticity executes as designed: 300 reserve neurons are activated and ≈ 13.2–13.9 k synapses created (150 retired) per seed, bringing the network from 1,065 to 1,365 neurons. On the main recode run the gain-50 `structural_on` condition is the study's largest decoder accuracy (**0.833**), and by the decomposition above it is attributable to the recruitment rule, not the

dopamine rule. The growth-matched controls in Section 4.3b, however, show that this "growth effect" is a decoder-capacity effect rather than a learned, label-guided prototype, and that the absolute 0.833 does not reproduce under a different condition ordering (0.750).

4.3b Growth-matched controls demote the growth interpretation

The `structural_on` gain had three possible explanations: the recruitment rule learns a class code; it merely expands the decoder's feature budget; or its label-guided category inheritance constructs a class prototype. The controls in `configs/odor-recode-50-controls.yaml` separate them, run at 10 seeds (`runs/recode-50-controls-merged`, gain 50, 600 trials): growth with random recruit categories, growth with no reward/label guidance at all (random anchor, random inputs, random categories), and a no-growth decoder given the same number of extra feature dimensions as growth adds (300 fixed random projections of the Kenyon-cell population).

Table 3: **Table 3.** Growth-matched control run (`runs/recode-50-controls-merged`, gain 50, 10 seeds, chance 0.25, 95 % bootstrap CIs over seeds). The internally-paired contrasts within this run are the quantities of interest; absolute numbers differ slightly from the main recode run because per-condition random state is not perfectly isolated (Section 5.4).

condition	decoder acc (95 % CI)	online argmax
untrained	0.330 [0.265, 0.390]	0.250
trained	0.418 [0.367, 0.468]	0.235
structural_off	0.378 [0.340, 0.417]	0.260
structural_on	**0.750** [0.672, 0.827]	0.250
structural_random_category (control)	0.707 [0.602, 0.802]	0.272
structural_unguided (control)	0.785 [0.692, 0.867]	0.262
dopamine_off	0.828 [0.720, 0.907]	0.250
dopamine_off_no_growth	0.432 [0.365, 0.497]	0.250
trained_extra_features (control)	**0.983** [0.973, 0.992]	0.248

Table 4: **Table 4.** Paired per-seed contrasts for the control run (Wilcoxon signed-rank, n = 10). Positive Δ means the first condition is higher.

contrast	Δ accuracy	p	reading
structural_on – structural_off	+0.372	0.002	growth beats no growth
structural_on – trained	+0.332	0.002	growth beats plain training
structural_on – structural_unguided	−0.035	0.75	label guidance not needed
structural_on – structural_random_category	+0.043	0.61	category inheritance not needed
structural_on – dopamine_off	−0.078	0.084	dopamine off is not worse
trained – trained_extra_features	−0.565	0.002	extra decoder capacity dominates
trained – untrained	+0.088	0.023	small training increment over the floor

Three conclusions:

- **Growth beats no-growth, robustly** (`structural_on` 0.750 vs `structural_off` 0.378 and `trained` 0.418, +0.37, $p = 0.002$). The growth switch, not dopamine, moves the decoder accuracy.
- **Label guidance is not the mechanism.** Removing all reward/label guidance (`structural_unguided` 0.785) or breaking category inheritance (`structural_random_category` 0.707) leaves accuracy statistically unchanged ($p = 0.75$ and 0.61). The growth benefit is therefore not the label-guided prototype construction we flagged as the central risk.
- **Decoder capacity explains it.** A **no-growth** decoder given 300 random Kenyon-cell projections reaches **0.983** — above every growth condition and 0.565 above `trained` ($p = 0.002$). Since growth likewise just adds 300 decoder feature dimensions, the honest reading is that growth improves *access* to an already decodable Kenyon-cell code, not that it learns one.

The controls were designed with the pre-registered expectation that `structural_on` would beat the unguided/random-category controls and that `trained_extra_features` would stay near the floor; both predictions failed, and we report that failure as the result. Dopamine remains non-contributory: `dopamine_off` (0.828) is not worse than `structural_on` (0.750, $p = 0.084$). The online argmax stays at chance (0.235–0.272) in every condition.

4.4 The gain restores the Kenyon-cell class code

The representation probes show the recode works by restoring the code, not by a side-effect on the readout (Table 5). At the physiological operating point the class code is lost at the `ALPN → Kenyon cell` transform: the stimulus decodes perfectly (input probe 1.000) but the Kenyon-cell population

does not (probe 0.255), and no readout recovers it — a supervised-pretrained readout that fits the Kenyon-cell features to 1.000 still generalises at chance (0.268), and a task with trivially separable categories remains at chance. Raising the gain restores the Kenyon-cell code: CV decodability rises to **0.950** at gain 20 and **1.000** at gain 50.

Table 5: **Table 5.** Kenyon-cell representation probe (k-fold CV accuracy) versus the `ALPN → Kenyon cell` gain.

ALPN → Kenyon gain	input probe	Kenyon-cell probe	trained held-out acc
1 (physiological)	1.000	0.255	0.262
20	1.000	0.950	0.363
50	1.000	1.000	0.418

The gating is causal: the probe locates where the code is lost, and the single intervention that restores it is the one that enables learning. This localisation — not the absolute accuracy — is the study's most robust contribution.

Table 6: **Table 6.** Condition definitions. The ablation is only interpretable if each condition's switches are stated, so this table is exhaustive: `dop` = the dopamine-gated weight update; `growth` = structural recruitment; `shuffle` = training labels shuffled; `pretrain` = readout initialised from a supervised fit; `decoder features` = what the supervised linear decoder reads (`4` = the four MBON counts; `+N` = plus N recruited units).

condition	trained	dop	growth	shuffle	pretrain	decoder features
untrained	no	—	off	no	no	4
trained	yes	on	off	no	no	4
structural_off (≡ trained)	yes	on	off	no	no	4
structural_on	yes	on	on	no	no	4 + recruits (≈300)
dopamine_off	yes	off	on	no	no	4 + recruits (≈300)
dopamine_off_no_growth	yes	off	off	no	no	4
shuffled_labels	yes	on	on	yes	no	4 + recruits (≈300)
shuffled_labels_no_growth	yes	on	off	yes	no	4
trained_pretrained	yes	on	off	no	yes	4

Growth-matched controls. The `structural_on` number rests on a recruitment rule that is *guided by the network's own predictions* (Section 5.2): recruits inherit the winning unit's category, and the recruited units become extra decoder feature dimensions. To separate learning from label-guided prototype construction and from a pure feature-dimension effect, three matched controls are implemented in `configs/odor-recode-50-controls.yaml` and **run at 10 seeds** (Section 4.3b, Tables 3–4):

- `structural_random_category` — growth on, but each recruit is assigned a **random** category instead of inheriting the anchor's (`structural.recruitment.category_guidance: false`).
- `structural_unguided` — growth on with a random anchor, uniformly random inputs, and random categories: no reward/label guidance of network structure at all, matched in recruit count and fan-in statistics.
- `trained_extra_features` — no growth, but the decoder is given the same number of extra (random Kenyon-cell projection) feature dimensions that growth adds (`evaluation.extra_random_features: 300`), isolating decoder capacity.

The pre-registered reading was: if `structural_on` beats the unguided and random-category controls and `trained_extra_features` stays near the untrained floor, growth does more than expand the decoder's feature budget. **The first half held and the second failed**: growth beats no-growth but is unchanged by removing label guidance, and `trained_extra_features` reaches 0.983, above growth. The 0.833 number is therefore not evidence of circuit learning; it is a decoder-capacity effect.

4.5 The above-floor signal is not leakage

A recoded representation is decodable even without learning, so the shuffled-label control sits at the untrained floor (0.330 at gain 20) rather than at raw chance; this is expected and is why the leakage margin is untrained-relative. We confirmed no leaking pipeline with two isolation controls (`runs/recode-v2-extra-merged`, `runs/recode-50-extra-merged`) in which labels are shuffled *and* structural growth is switched off: both sit at or below chance — **0.230** ($p = 0.75$) at gain 20 and **0.233** ($p = 0.72$) at gain 50 — with no leakage warning. The residual above chance in the growth-on conditions is attributable to structural recruitment, not to the evaluation pipeline leaking label information.

4.6 A decoder-fit leak, caught and fixed

An earlier recode attempt (`recode-v1`) produced a trained accuracy of 0.363 but its shuffled-label control scored **0.615**, raising the leakage warning and invalidating the run. The cause was a harness bug: the supervised decoder was fitted on the dataset's (true) labels in *every* condition, including the shuffled control, so the control had a classifier reading true labels off the network's training outputs — a mapping the network was never taught. The fix fits the decoder on the labels each trial was actually *presented* with. This is a no-op for every non-shuffled condition (there the presented labels are the dataset labels), so all other numbers stand; the shuffled control dropped to the untrained floor. A regression test reproduces the leak and guards against it.

5. Discussion

5.1 The input transform gates learning

The gating mechanism is precise. At the physiological operating point the class code is lost at the `ALPN → Kenyon cell` transform, and this is not the output drive (the MBON stage is not silent), not the input encoding (the input probe is 1.000), and not the readout or its credit rule (a supervised-pretrained readout that fits the Kenyon-cell features to 1.000 still generalises at chance, and a separable task is equally at chance). The Kenyon-cell population is where the class code is destroyed (probe 0.255). Raising that transform's gain restores the code (probe 0.950 ⟶ 1.000). In this calibrated regime the `ALPN → Kenyon cell` transform — not the plastic readout — is the gate on learning, and that gate can be opened by a single, interpretable parameter.

This sensitivity to input gain and sparsity is what expansion-layer and random-projection accounts of Kenyon-cell coding predict: when the projection gain is too low relative to the spontaneous drive, the Kenyon-cell expansion collapses toward a low-dimensional, class-agnostic code, and widening or rebalancing the projection restores a linearly separable representation. The result here is a calibrated-instance of that principle rather than a new mechanism, which is why it is reported as a diagnostic localisation.

5.2 What the largest number actually rests on: decoder capacity

The best result (0.833 main run; 0.750 matched-control run) is produced by the growth-on conditions, and the structural-recruitment rule is a **supervised, exploratory** mechanism: a reserve neuron is recruited near the most successful readout unit and *inherits that unit's category* and most of its inputs, and the recruited units become extra decoder feature dimensions. The natural worry was that the rule constructs a class prototype rather than learning a code. The controls run in Section 4.3b settle it, and they go against the growth interpretation:

- The gain is **unchanged when all label guidance is removed** (`structural_unguided` 0.785 vs `structural_on` 0.750, p = 0.75) and when category inheritance is broken (`structural_random_category` 0.707, p = 0.61), so label-guided prototype construction is not the mechanism.
- A **no-growth decoder given 300 random Kenyon-cell projections reaches 0.983** (p = 0.002 vs `trained`), above every growth condition. Growth adds the same number of feature dimensions, so its benefit is best read as expanded decoder access to a Kenyon-cell code the recode already made decodable — a **decoder-capacity / feature-expansion effect**, not a learned structure.
- The online MBON-argmax readout never leaves chance (Tables 1 and 3), so growth adds decoder-readable structure without producing an online class decision.

The honest summary is therefore narrower than a learning claim: under the recode the Kenyon-cell population carries a class code that an offline supervised decoder can exploit, and structural growth is one — sub-optimal — way to give that decoder more to work with. The plain dopamine-gated trained condition is only marginally above the untrained floor, and dopamine remains irrelevant throughout. The durable positive result is the causal localisation of the gate to the `ALPN → Kenyon cell` transform (Section 5.1), not the growth number.

5.3 The readout and the operating point do part of the work

Two departures inflate the result relative to a claim about online learning. The reported accuracy uses a **supervised linear decoder fitted after training** on a fixed feature pass; the online MBON-argmax readout was also measured and stays at chance (0.235–0.272 across all conditions except `trained_pretrained`, Tables 1 and 3), so the headline number is the post-hoc decoder's representation-quality readout, not the plasticity rule's online decision. And the recode operates at a **100 Hz calibration ceiling**, far outside the physiological 0.5–15 Hz band. Both are diagnostic choices that isolate the transform as the bottleneck; neither should be read as a claim about the circuit *in vivo*.

5.4 A wiring caveat and a reproducibility caveat

The real `DAN → Kenyon cell` edges are extracted but are **not** simulated: `build_network` wires consecutive stage pairs, within-stage recurrence, and the plastic `mushroom body → output` pair. Because `reinforcement` is configured after `mushroom_body`, `reinforcement → mushroom_body` is a backward pair and is dropped; the reward is applied as a global scalar gated by the real DANs' spike counts rather than through the real synapses. The config's "real DAN ⟶ Kenyon cell" pathway is therefore not the one trained; reordering the stages would exercise those edges.

Separately, `trained` and `structural_off`, which share the identical configuration, do not produce identical results (0.363 vs 0.370; 0.418 vs 0.435). The small differences are consistent with residual run-order dependence in RNG consumption across conditions rather than with a real effect, but they indicate that per-condition random state is not perfectly isolated. We report them so the identical-condition replicates are not mistaken for a contrast.

5.5 Limitations

- Task and network are small by design (CPU-only reproducibility) and the stimuli are low-dimensional prototype patterns, so the results speak to this calibrated regime and this rule, not to mushroom-body or word learning in general.
- The growth number is explained by decoder capacity: the growth-matched controls (Section 4.3b) show the gain is unchanged without label guidance and is exceeded by random features with no growth, so the largest numbers are not evidence of circuit learning.
- Absolute numbers vary with condition ordering (per-condition random state is not perfectly isolated; Section 5.4), so only internally-paired contrasts are interpreted.
- The reported readout is a supervised post-training decoder; the recode leaves the physiological band (Section 5.3).
- All conditions share a single calibration per run; a broader sweep of operating points was not performed. The recode is reported at two gains (20 and 50), so the claimed monotone dependence on gain is not established by more than two points; a sweep over gains and calibrated scales is future work.

6. Conclusion

A real, synapse-level FlyWire subcircuit with zero artificial edges carries a linearly decodable word-category code once a single input transform is recoded: raising the `ALPN → Kenyon cell`

gain restores a Kenyon-cell class code that the physiological operating point destroys (input probe 1.000, Kenyon-cell probe 0.255 → 0.95–1.00). We are deliberately conservative about the resulting accuracy. The untrained network already sits at 0.33, the plain dopamine-gated trained condition is only marginally above it ($p = 0.02$), and the online MBON-argmax readout is at chance throughout.

The largest supervised-decoder numbers are produced by structural growth (growth-on 0.833 main run / 0.750 matched-control run, exceeding growth-off conditions by +0.37–0.40, paired $p < 0.001$; switching dopamine off changes nothing). The growth-matched controls show what those numbers are: the gain is unchanged when all label guidance is removed (`structural_unguided` 0.785, $p = 0.75$) or category inheritance is broken ($p = 0.61$), and it is exceeded by a no-growth decoder given 300 random Kenyon-cell projections (0.983, $p = 0.002$). The growth effect is therefore a decoder-capacity effect — expanded access to an already decodable representation — not a learned, label-guided prototype, and not an online learned behaviour.

We report this as the study's outcome, including the pre-registered predictions that the controls falsified. Its durable contribution is (i) a causal localisation of the class-code bottleneck to the `ALPN → Kenyon cell` transform, (ii) a self-corrected decoder-fit leak and its regression test, (iii) a set of released growth-matched controls that demote the paper's own headline, and (iv) a reproducible pipeline in which all of these claims can be inspected and challenged.

Data and code availability

Code: `ngiaved/fly-word-categorization` (FlyWire data is CC-BY-NC and downloaded by `flyread fetch-data`). The recoded runs are archived under `runs/recode-v2-merged/` (gain 20) and `runs/recode-50-v2-merged/` (gain 50); the growth-matched control run is `runs/recode-50-controls-merged/` (gain 50, 10 seeds; Section 4.3b); the shuffled no-growth isolation controls are under `runs/recode-v2-extra-merged/` and `runs/recode-50-extra-merged/`. The physiological-operating-point reference runs are `runs/odor-v1-merged/` (baseline) and `runs/pctrl-v1-merged/`, `runs/pctrl-sep-merged/` (probes and separable-task control). Every archived run carries its manifest. The growth-matched control configuration is released at `configs/odor-recode-50-controls.yaml`.

References

The following are the key background works; bibliographic details should be verified against the primary sources before submission.

1. Aso Y, Hattori D, Yu Y, et al. (2014). The neuronal architecture of the mushroom body provides a logic for associative learning. *eLife* 3:e04577.
2. Dorkenwald S, Matsliah A, Sterling AR, et al. (2024). Neuronal wiring diagram of an adult brain. *Nature* 634:124–138.
3. Hebb DO (1949). *The Organization of Behavior*. Wiley.
4. Hige T, Aso Y, Rubin GM, Turner GC (2015). Plasticity-driven individualization of olfactory coding in mushroom body output neurons. *Nature* 526(7572):258–262.

5. Scheffer LK, Xu CS, Januszewski M, et al. (2020). A connectome and analysis of the adult *Drosophila* central brain. *eLife* 9:e57443.
6. Schlegel P, Yin Y, Bates AS, et al. (2024). Whole-brain annotation and multi-connectome cell typing of *Drosophila*. *Nature* 634:139–152.
7. Schultz W, Dayan P, Montague PR (1997). A neural substrate of prediction and reward. *Science* 275:1593–1599.
8. Stimberg M, Brette R, Goodman DFM (2019). Brian 2, an intuitive and efficient neural simulator. *eLife* 8:e47314.
9. Zheng Z, Lauritzen JS, Perlman E, et al. (2018). A complete electron microscopy volume of the brain of adult *Drosophila melanogaster*. *Cell* 174(3):730–743.

Appendix A. Reproducing the runs

```
python -m pip install -r requirements.lock.txt
python -m pip install -e .
python -m flyread fetch-data

# main result (recode)
python -m flyread --config configs/odor-recode.yaml    --run-id recode-v2
sweep
python -m flyread --config configs/odor-recode-50.yaml --run-id recode-50-v2
sweep

# growth-matched controls (run; Sections 4.3b, 5.2)
python -m flyread --config configs/odor-recode-50-controls.yaml \
    --run-id recode-50-controls sweep

# reference at the physiological operating point
python -m flyread sweep
python -m flyread --config configs/default.yaml --run-id pctrl-v1 sweep
python -m flyread --config configs/odor-separable.yaml --run-id pctrl-sep sweep
```

The `sweep` command runs one `evaluate` process per seed, resumes completed (seed, condition) pairs from the checkpoint, and merges the shards into a single multi-seed report. Every run writes a manifest (config, seed state, checksums, calibration, subcircuit statistics, codegen target, hardware, git commit). The formal 10-seed run needs ≈ 126 h of single-core compute (≈ 13.8 h wall-clock on 12 cores) and fits in 16 GB; the reported runs are CPU-only with Cython-compiled kernels. The growth-matched control run (9 conditions × 10 seeds) took ≈ 242 h of single-core compute (≈ 28.6 h wall-clock on 10 workers).